

Università degli Studi di Perugia

Dipartimento di Informatica / Ingegneria Informatica

Corso di Intelligenza Artificiale

# Analisi e Digitalizzazione di Mappe Geopolitiche Tramite Computer Vision e Deep Learning

Anno Accademico 2025/2026

*Studenti:*

Romeo Miscio 364672

Tommaso Moschini —

Federico Tabuani —

*Docente:*

Prof.ssa Valentina Poggiani

Ultima modifica: 05/06/2026

# Indice

|          |                                                                      |          |
|----------|----------------------------------------------------------------------|----------|
| <b>1</b> | <b>Definizione del Dominio: Navigazione e Mappatura Territoriale</b> | <b>2</b> |
| 1.1      | Architettura della Mappa e Discretizzazione . . . . .                | 2        |
| 1.2      | Logica di Movimento e Modello dei Costi . . . . .                    | 2        |
| 1.3      | Classificazione e Tipologia dei Territori . . . . .                  | 3        |
| <b>2</b> | <b>Architettura del Software e Scelte Progettuali</b>                | <b>3</b> |
| <b>3</b> | <b>Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione</b> | <b>4</b> |
| 3.1      | Trattamento del Tratto e Rimozione dei Quadretti . . . . .           | 4        |
| 3.2      | Gestione dell'Illuminazione . . . . .                                | 4        |
| <b>4</b> | <b>Il Modulo ML: Riconoscimento dei Caratteri</b>                    | <b>4</b> |
| 4.1      | Standardizzazione dell'Input . . . . .                               | 5        |
| <b>5</b> | <b>L'Hub Master: Integrazione e Logica di Sistema</b>                | <b>5</b> |
| 5.1      | Classificazione tramite Analisi delle Aree . . . . .                 | 5        |
| 5.2      | Topologia degli Stati e Colorazione . . . . .                        | 5        |

# 1 Definizione del Dominio: Navigazione e Mappatura Territoriale

Il presente progetto si pone l'obiettivo di modellare un sistema di navigazione per un agente operante in un ambiente discretizzato, caratterizzato da territori frazionati e costi di transito variabili[cite: 2].

## 1.1 Architettura della Mappa e Discretizzazione

L'ambiente è rappresentato come una matrice di celle, dove la granularità della griglia determina la precisione della rappresentazione geografica[cite: 2]:

- **Rappresentazione degli Stati:** Uno stato (o regione) può coincidere con una singola cella o essere costituito da un cluster di più celle contigue per rappresentare estensioni territoriali maggiori[cite: 2].
- **Segmentazione Cromatica:** Al fine di distinguere topologicamente le diverse regioni, ogni stato è caratterizzato da un colore differente rispetto a quelli adiacenti[cite: 2]. Questo garantisce una separazione netta dei confini durante la fase di analisi dell'immagine[cite: 2].
- **Processo di Pixellazione:** La definizione dei confini avviene attraverso una procedura di pixellazione che mappa l'input visivo sulla griglia di navigazione, definendo l'appartenenza di ogni coordinata  $(x, y)$  a uno specifico stato[cite: 2].

## 1.2 Logica di Movimento e Modello dei Costi

Il movimento dell'agente è vincolato a spostamenti tra celle adiacenti lungo le direzioni cardinali (Nord, Sud, Est, Ovest)[cite: 2]. La funzione di costo associata alle azioni di navigazione è così definita[cite: 2]:

- **Costo Base di Spostamento:** Il transito tra due celle adiacenti ha un costo base unitario pari a 1[cite: 2].
- **Penalità di Confine:** L'attraversamento di un confine tra due stati distinti comporta un onere aggiuntivo di +1 sul costo totale dello spostamento (risultando quindi in un costo totale pari a 2)[cite: 2].
- **Penalità Territorio Nemico:** L'ingresso in uno stato ostile applica un malus aggiuntivo istantaneo al costo del tragitto[cite: 2]. Tale malus è calcolato moltiplicando il livello di pericolosità del territorio nemico (con valori compresi tra 1 e 9) per un parametro di calibrazione  $X$  (da definire in fase di configurazione)[cite: 2].

### Formalizzazione Matematica

Dato un percorso che attraversa  $n$  celle, interseca  $k$  confini di stato e prevede l'ingresso in  $m$  stati nemici (ciascuno caratterizzato da un livello di pericolosità  $L_i$ ), il costo totale  $C$  è espresso dalla seguente formula[cite: 2]:

$$C = n + k + \sum_{i=1}^m (L_i \cdot X) \quad (1)$$

*Esempio pratico:* Uno spostamento attraverso 6 celle all'interno dello stesso stato alleato ha un costo pari a 6[cite: 2]. Se lo stesso spostamento prevede l'attraversamento di un confine per entrare in uno stato nemico con livello di pericolosità 4, il calcolo del costo diverrà: 6 (costo celle) + 1 (costo confine) +  $4 \cdot X$  (malus nemico)[cite: 2]. Il costo totale sarà quindi pari a  $7 + 4X$ [cite: 2].

### 1.3 Classificazione e Tipologia dei Territori

In fase di acquisizione, ogni cella viene analizzata e classificata per definirne le proprietà fisiche e tattiche[cite: 2]. Invece di utilizzare nomenclature estese, il sistema adotta una codifica alfanumerica ottimizzata[cite: 2]:

| Identificatore | Tipologia         | Proprietà                                                |
|----------------|-------------------|----------------------------------------------------------|
| <b>S</b>       | Start             | Punto di origine dell'agente[cite: 2].                   |
| <b>W</b>       | Win               | Obiettivo finale o destinazione del percorso[cite: 2].   |
| <b>A</b>       | Alleato           | Territorio amichevole con transito standard[cite: 2].    |
| <b>X</b>       | Confine Naturale  | Ostacolo insormontabile (non attraversabile)[cite: 2].   |
| <b>1 - 9</b>   | Territorio Nemico | Rappresenta il grado di pericolosità crescente[cite: 2]. |

Tabella 1: Codifica alfanumerica e tipologia dei territori acquisiti[cite: 2]

I valori numerici relativi alla pericolosità dei territori nemici sono direttamente integrati nella funzione di costo globale[cite: 2]. Questo approccio parametrico (modulabile tramite la variabile  $X$ ) permette all'algoritmo di ricerca di valutare percorsi più o meno prudenti, bilanciando in modo dinamico la brevità del tragitto con l'esposizione al rischio[cite: 2].

## 2 Architettura del Software e Scelte Progettuali

L'architettura del sistema è stata sviluppata seguendo un approccio modulare, ispirato ai principi dei microservizi, al fine di garantire scalabilità e facilità di debugging[cite: 1]. Inizialmente concepito come uno script monolitico, il progetto è stato rifattorizzato in tre moduli distinti per evitare conflitti tra le librerie di elaborazione delle immagini e quelle di apprendimento automatico[cite: 1].

La struttura finale si compone di:

- **Hub Centrale** (`hub_master.py`): Agisce come controller principale del flusso di esecuzione[cite: 1].
- **Modulo Digitalizzatore** (`modulo_digitalizzatore.py`): Specializzato nelle operazioni di pre-elaborazione e computer vision[cite: 1].
- **Modulo ML** (`modulo_ml.py`): Deputato al riconoscimento dei caratteri tramite reti neurali[cite: 1].

Questa separazione delle responsabilità permette di isolare eventuali errori logici o di classificazione, garantendo che interventi sulla rete neurale non compromettano la gestione geometrica dei confini[cite: 1].

### 3 Il Modulo Digitalizzatore: Computer Vision e Pre-elaborazione

Il modulo `modulo_digitalizzatore.py` ha il compito di trasformare un'acquisizione fotografica grezza, potenzialmente degradata da rumore ottico o condizioni di illuminazione non ideali, in una matrice binaria pulita[cite: 1].

#### 3.1 Trattamento del Tratto e Rimozione dei Quadretti

Uno dei problemi principali affrontati è stata la presenza della griglia dei quadretti del foglio, il cui spessore era paragonabile a quello del tratto della penna[cite: 1]. Per preservare l'integrità dei confini durante la rimozione del rumore, è stata adottata la seguente pipeline[cite: 1]:

1. **Erosione Morfologica:** Utilizzando la funzione `cv2.erode` con un kernel ridotto, è stato possibile espandere i pixel neri, ispessendo preventivamente il tratto della penna[cite: 1].
2. **Filtraggio Mediano:** È stato applicato un filtro `cv2.medianBlur` con intensità 11[cite: 1]. Tale operazione sfrutta la media dei pixel adiacenti per assorbire elementi sottili (i quadretti) nello sfondo, mantenendo intatti i confini precedentemente ispessiti[cite: 1].

#### 3.2 Gestione dell'Illuminazione

Per ovviare al problema delle ombre proiettate durante l'acquisizione, è stato implementato un algoritmo di **Soglia Adattiva** (`cv2.adaptiveThreshold`)[cite: 1]. Calcolando il contrasto su finestre locali di  $31 \times 31$  pixel, il sistema è in grado di distinguere accuratamente tra variazioni cromatiche dovute a ombreggiature e l'effettivo tratto grafico[cite: 1].

È importante sottolineare la rimozione della funzione di pulizia globale (precedentemente denominata "Vacuum Cleaner")[cite: 1]. Sebbene efficace nel rimuovere artefatti esterni, tale funzione avrebbe eliminato anche i caratteri interni agli stati, fondamentali per l'etichettatura semantica della mappa[cite: 1].

### 4 Il Modulo ML: Riconoscimento dei Caratteri

Il modulo `modulo_ml.py` funge da wrapper per una Rete Neurale Convoluzionale (CNN)[cite: 1]. Per ottimizzare le prestazioni, il modello viene caricato in memoria esclusivamente all'avvio del programma[cite: 1].

## 4.1 Standardizzazione dell'Input

La rete è stata addestrata sul dataset **EMNIST**, che impone input di dimensioni  $28 \times 28$  pixel[cite: 1]. Il processo di normalizzazione dei ritagli forniti dall'Hub avviene secondo i seguenti criteri[cite: 1]:

- Ridimensionamento del ritaglio mantenendo il rapporto d'aspetto, con lato massimo fissato a 20 pixel[cite: 1].
- Posizionamento centrale dell'immagine all'interno di una tela nera di  $28 \times 28$  pixel[cite: 1].

Grazie alla precisione dell'estrazione geometrica effettuata dall'Hub, è stato possibile eliminare l'uso di librerie esterne come **scipy** per la ricerca delle forme, riducendo significativamente il carico computazionale[cite: 1].

## 5 L'Hub Master: Integrazione e Logica di Sistema

L'Hub Master coordina l'interazione tra i moduli e risolve le criticità topologiche e algoritmiche emerse durante lo sviluppo[cite: 1].

### 5.1 Classificazione tramite Analisi delle Aree

Per mitigare l'esplosione di falsi positivi derivanti dai residui della digitalizzazione, è stata introdotta una classificazione euristica basata sulla funzione **connectedComponents**[cite: 1]. Gli elementi vengono filtrati in base all'area (in pixel)[cite: 1]:

- **Area < 400**: Classificata come rumore o "polvere" e scartata[cite: 1].
- **400 < Area < 15.000**: Identificata come potenziale carattere alfanumerico e inviata al modulo ML[cite: 1].
- **Area > 15.000**: Identificata come confine geografico[cite: 1].

### 5.2 Topologia degli Stati e Colorazione

Un problema critico è stato l'errata interpretazione dei confini interni utilizzando la funzione **cv2.RETR\_EXTERNAL**, che portava alla rilevazione del solo perimetro esterno[cite: 1]. La soluzione adottata prevede l'inversione dello spazio di ricerca: l'algoritmo identifica gli spazi bianchi vuoti come regioni isolate (stati) anziché concentrarsi sulle linee nere[cite: 1].

Attraverso l'uso di identificativi univoci (ID) e dell'operatore modulo applicato a una palette di colori, il sistema assegna colori distinti a nazioni adiacenti[cite: 1]. Infine, l'associazione tra etichette testuali e regioni geometriche è garantita dal test **Point in Polygon** (**cv2.pointPolygonTest**), che verifica l'appartenenza delle coordinate della lettera al poligono dello stato corrispondente[cite: 1].